Basic Overview:

This application is a program designed to curate wines for you based on an initial survey. There is an element of randomness to the decision algorithm to allow for unique results on repeat usage. Based on your response to the recommendations the algorithm is further adjusted to filter the results of your search on subsequent usage.

Running the Program:

I have included a number of files. There are two avenues to run my program:

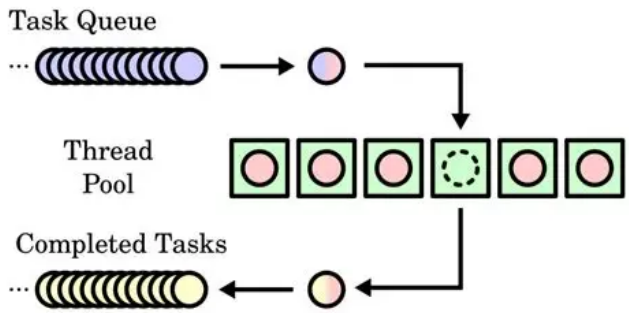
1. Download python and then use the pip installer to download the following packages. Run main and you are good to go. This is a lot of work, especially for someone who isn't tech savvy. I recommend option two.

   httpx
   numpy
   scikit-learn
   Pillow
   Tkinter
   Vinofyi api

2. I have packaged my program conveniently into a .exe so that you don't have to install anything. Your antivirus will likely flag this because it's a random program created by me. You will have to bypass this warning, and once you do you should not need to install anything as long as you are running the program on a 64 bit (90% of machines today) windows machine. Simply run the program Curator.exe and it builds the necessary file directories for you.

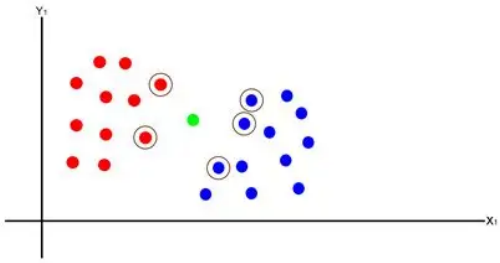
Definitions:

- **Thread** - A thread is a singular logical processor. Modern computers contain multiple processors. Threaded programs leverage more than one of these processors to work on things simultaneously. This is like doing two loads of laundry at the same time by having one in the dryer and one in the washer as opposed to only using either the washer or the dryer.



- **Nearest Neighbors** - There are multiple different algorithms under this umbrella, but all use the same fundamental logic. If I'm a point on a graph, what points are closest to me? What can I discern from the elements closest to me (or furthest away)? This is a trivial question to conceptualize when exploring in 1-2 dimensions. Highschool math has you figure out the pythagorean distance between two points. This becomes harder when trying to explore larger datasets where you may even have hundreds of different dimensions.

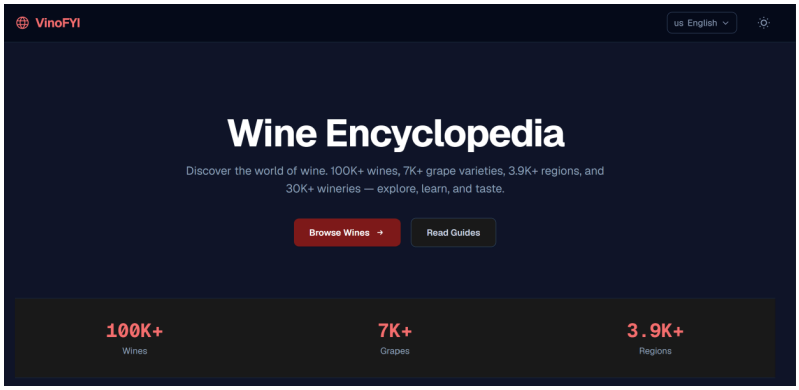
  For my code, sweetness, acidity, abv, etc are all different dimensions making my data seven dimensional. Nearest neighbors explores this seven dimensional space and uses a euclidean distance measurement (you don't need to understand what this is, just think of it as pythagorean extended to more dimensions) to find the points closest to our starting point. The logic is that if you treat the wine characteristics as dimensions, the wines in closest proximity to our starting point (the user's survey) should be the most similar to the user's preferences. I'll explore some of the implications of this later.



- **Attribute** - a nearest neighbors attribute is synonymous with a dimension. For my code these attributes are the characteristics of a wine which I define below.

```
arr_samples.append(np.array([
    self.get_value("type",wine.get("type")),
    self.get_value("acidity",wine.get("acidity")),
    self.get_value("sweetness",wine.get("sweetness")),
    self.get_value("abv",wine.get("abv")),
    self.get_value("body",wine.get("body")),
    self.get_value("world",wine.get("world")),
    self.get_value("liked_grapes",common_grapes)
]))
```

- **Sample** - In machine learning a sample refers to a singular data point. For me, a sample is either a user survey or a wine.
- **Database** - An online repository which collects some kind of data which can be retrieved by a user. Some databases are publicly accessible while others require authorization. VinoFYI is a public wine database used by my program. It is linked below in my sources if you want to learn more.



- **Query** - A query is a call to a database to obtain information. In this case these queries are returned in the JSON format. Note that the JSON example below is the output of my fetch which adds new attributes and adjusts formatting from the raw JSON obtained from the database.


```
{
    "wine": "Atlantis Fe203 1A Syrah",
    "grapes": [
        "syrah"
    ],
    "sweetness": "Unknown",
    "type": "Red",
    "acidity": "High",
    "abv": "High",
    "country_name": "United States",
    "world": "New World",
    "body": "Full-bodied",
    "winery_name": "Sine Qua Non",
    "region_name": "California"
},
```


- **Scikit-learn** - This is a “library” (basically an extension) for python which adds machine learning features to Python. You can learn about the library linked below.

Files:

main.py - Main program spin up and setup tkinter
load.py - Loading screen visual and starts fetch
fetch.py - Queries and compiles data into my attribute format. Uses threads to speed up the process.
quiz.py - Class for easily creating tkinter quiz
recommender.py - Class which deals with formatting user and database wine samples into my internal format for scikit-learn (learning algorithm tool).
app_paths.py - Streamlines data saving locations for easier app building

past_recommendations.json - Location saving past recommendations and how the user rated them
master_wine_data.json - Holds the result of the massive query on program start

Red_wine.png - Public domain image of a glass of red wine.

Nearest Neighbors Attributes:

"type"
"acidity"
"sweetness"
"abv"
"body"
"world"
"liked_grapes"

Process:

1. Query runs which grabs wines from a wine database to build a catalog of wines
2. Threads randomly select wines from the catalog and build JSON elements using the attributes as is or conditional logic to create new attributes. Ex: Sweetness is not an attribute in the database, so the threads look for sweetness keywords in the names of wines.
3. Main run loop and window spins up presenting the user a survey to assess their preferences.
4. Once the user submits their survey results the results are used to convert the database samples and user sample to a scikit-learn friendly numeric format using py.recommender.
5. Scikit Learn runs its nearest neighbors to get the 5 best recommendations (these are orders with 1 being the best and 5 being the worst.)
6. Scikit Learn runs nearest neighbors in one of the following ways to filter the previous nearest neighbors by past recommendations.
    a. If past_recommendations.json contains only **one** element: Run nearest neighbors with the **PAST** recommendation as input and **CURRENT** recommendations as target.
        i. If the past recommendation has a rating > 3 then give the recommendation **MOST** similar to the well rated past recommendation.
        ii. If the past recommendation has a rating < 3 then give the recommendation **LEAST** similar to the poorly rated past recommendation.
        iii. If the past recommendation has a rating = 3 then **IGNORE** this step and give the recommendation Scikit Learn identified as the overall best fit in step 5.
    b. If past_recommendations.json contains **TWO** elements: Run nearest neighbors 3 times once for each of the **CURRENT** recommendations as input and the **PAST** recommendations as targets.
        i. Assign each recommendation the rating of whichever of the two neighbors is the **MOST** similar. Whichever recommendation has the highest rating, select it. In the probable event of a tie; choose the element from the tie that is defined as best by step 5.
    c. For **ANY OTHER NUMBER** of elements: Run nearest neighbors 3 times once for each of the **CURRENT** recommendations as input and the **PAST** recommendations as targets.

    i. Assign each current recommendation the sum of the two **MOST** similar wine's ratings. Recommend the user the wine whose sum is the **GREATEST** following this process. In the probable event of a tie; choose the element from the tie that is defined as best by step 5.
7. Creates two new windows. One opens the recommendation in a browser window and the other prompts the user to rate the accuracy of this recommendation.
8. Once rated the rating window closes giving the user the opportunity to ask for another recommendation with the same or different parameters. Subsequent recommendations here or in new sessions will be filtered by the users past ratings. The user has the option to reset this history using a button in the main window.

Conclusions:

This was my first coding project which utilized significant AI assistance through development. This was a new way of working which reflects industry trends. I learned a lot about what this process looks like as well as new coding techniques which the agents used that I was not aware of before. I learned how to utilize Scikit-learn which I leveraged heavily for its algorithms as well as Tkinter for my visuals. This was also my first time doing object oriented Python.

If I were to do this project again, I think I would use a different library for my visuals. Tkinter was easy and straightforward, but it doesn't look very polished and professional. If I had more time I would use a backend/frontend full stack approach. Tkinter was straight forward for the allotted time. My focus for this project was the underlying algorithm and data preprocessing.

I am very happy with the outcomes for this project. The work on this project was a culmination of weeks of development and iteration starting back in September. The Github backup is linked below if you want to look at my progress logs. I hope that you found learning about this program interesting. Go curating!

The project GIT: [Commits · Brick1026/Wines-of-The-World]

Sources:

- **Images:**
    - https://en.wikipedia.org/wiki/Thread_pool
    - https://www.codershood.info/2019/01/25/k-nearest-neighbors-k-nn-classifier-using-python-with-example/
- **Information**
    - https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/
- **Tools/APIs:**
    - https://github.com/fyipedia/vinofyi
    - https://github.com/numpy/numpy
    - https://github.com/python-pillow/Pillow
    - https://github.com/scikit-learn/scikit-learn